LeafLens

AI-Powered Leaf Disease Prediction System

Test Plan and Test Report

Prepared by

Abhijeet Kumar

DA24B001

1 Introduction

Testing was an important part of developing LeafLens. A model producing accurate predictions during experimentation does not automatically mean that the complete application will work reliably.

Users may upload invalid images, the backend may encounter unexpected errors, and monitoring components must be able to collect application metrics correctly.

This document describes how LeafLens was tested, which conditions were covered, and what was observed during the recorded test runs.

1.1 Testing Objectives

The main objectives were:

- Verify that the backend endpoints respond correctly.
- Confirm that valid images can be processed.
- Ensure that invalid uploads are rejected.
- Check that prediction failures are handled safely.
- Verify that Prometheus metrics are exposed and updated.
- Check that the containerised services start and communicate.
- Evaluate the trained model on a held-out test set.

2 Testing Scope

Testing was carried out at different levels, covering both the machine learning model and the application that serves its predictions.

Table 1: Testing levels and their purpose

Testing level	Purpose
Model evaluation	Measures classification performance on the held-out test dataset.
API testing	Checks endpoint behaviour, request handling, and response generation.
Input validation testing	Verifies that empty, invalid, and oversized uploads are rejected.
Metrics testing	Checks that application metrics are exposed and prediction activity is recorded.
Deployment testing	Checks that Docker Compose services start and communicate correctly.
Monitoring integration	Verifies that Prometheus can scrape the backend and Grafana can display the dashboard.

The recorded backend test suite contains eight automated tests. Deployment and monitoring checks were also performed separately.

3 Test Environment

The following environment was used during development and testing.

Table 2: Development and test environment

Component	Technology
Programming language	Python 3.12
Deep learning framework	PyTorch
Image processing	Pillow and Torchvision
Backend framework	FastAPI
Frontend framework	Streamlit
Testing framework	Pytest
Metrics collection	Prometheus
Monitoring dashboard	Grafana
Containerisation	Docker and Docker Compose

The backend tests were executed using the project's Python virtual environment. The containerised application was tested separately using Docker Compose.

4 Testing Workflow

The testing process follows a practical sequence. The model is evaluated first, followed by backend functional tests, monitoring checks, and deployment verification.

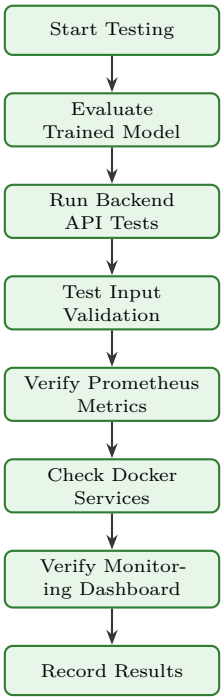


Figure 1: LeafLens testing workflow

5 Model Evaluation

Before integrating the model into the application, its classification performance was evaluated using the held-out test subset.

The model was trained using MobileNetV3-Large with pretrained ImageNet weights and fine-tuned for 38 plant disease and healthy-leaf classes.

Table 3: Model evaluation results

Metric	Result
Test samples	8,146
Correct predictions	8,108
Incorrect predictions	38
Accuracy	99.53%
Macro Precision	99.27%
Macro Recall	99.34%
Macro F1-score	99.28%

The model achieved 99.53% accuracy on the held-out test set. Macro precision, recall, and F1-score were also recorded to provide a view of performance across the different classes.

These results apply to the PlantVillage test split. They should not be interpreted as a guarantee of equivalent performance on photographs captured under uncontrolled field conditions.

6 Backend Functional Test Cases

The backend test suite contains eight test cases. They cover the main endpoints, prediction behaviour, input validation, and monitoring instrumentation.

Table 4: Detailed backend test cases

ID	Test case	Expected result
T01	Root endpoint	The root endpoint should respond successfully.
T02	Health endpoint	The health endpoint should return the expected health status.
T03	Successful prediction	A valid image should produce a prediction response.
T04	Empty upload	An empty file should be rejected.
T05	Invalid image	Unreadable image content should be rejected.
T06	Oversized upload	A file exceeding the 10 MB limit should be rejected.
T07	Metrics endpoint	The metrics endpoint should expose Prometheus data.
T08	Prediction metrics	A successful prediction should update the corresponding monitoring metrics.

6.1 Test Execution Results

The recorded backend test run completed successfully.

Table 5: Backend test execution summary

Result	Count
Total tests	8
Passed	8
Failed	0

8 / 8 tests passed

The test suite completed in approximately 4.74 seconds during the recorded run.

7 Acceptance Criteria

The following criteria were used to judge whether the main application functions were behaving as expected.

Table 6: Acceptance criteria

Area	Acceptance criterion
API availability	The backend should respond to its root and health endpoints.
Prediction	A valid image should produce a structured prediction response.
Input validation	Empty, invalid, and oversized images should be rejected.
Error handling	Unexpected prediction failures should be handled without exposing internal exception details.
Metrics	The metrics endpoint should expose application instrumentation.
Monitoring	Prometheus should be able to collect backend metrics, and Grafana should display the configured dashboard.
Deployment	The four Compose services should start and remain available for the local demonstration.

The eight automated backend tests passed. The deployment and monitoring criteria were also checked during the integration demonstration.

8 Deployment and Integration Testing

After testing the backend independently, the application was started using Docker Compose.

The deployment contains four services:

- FastAPI backend.
- Streamlit frontend.
- Prometheus metrics collector.
- Grafana monitoring dashboard.

Table 7: Docker Compose service verification

Service	Host Port	Observed state
Backend	8001	Running and healthy
Frontend	8501	Running
Prometheus	9092	Running
Grafana	3002	Running

The backend health check was successful, and the frontend was configured to communicate with the backend through the internal Docker network.

8.1 Prometheus Verification

Prometheus was checked to confirm that it could collect metrics from the backend.

The recorded target status showed both the Prometheus self-monitoring target and the LeafLens backend target as UP.

8.2 Grafana Verification

The Grafana dashboard was opened to verify that the configured Prometheus data source and dashboard were available.

A sample image prediction was performed during the demonstration. The dashboard reflected the prediction activity, including the predicted class and confidence.

9 Observed Application Behaviour

During the demonstration, the application successfully processed a sample leaf image.

The recorded prediction was:

Table 8: Sample prediction observation

Attribute	Observed value
Predicted class	Tomato Late Blight
Confidence	Approximately 99.9%
Prediction count	1
Prediction failures	0
HTTP errors per second	0

This observation confirmed that the prediction request reached the backend and that the corresponding activity appeared in the monitoring dashboard.

The sample prediction is an application demonstration, not a substitute for the complete held-out test-set evaluation.

10 Known Limitations and Future Testing

The current testing process verifies the main functional paths and local deployment behaviour. There are still areas that could be explored further.

- Load testing with a larger number of concurrent users.
- Testing with photographs captured under different lighting and outdoor conditions.
- Testing the application on additional hardware and deployment environments.
- Monitoring model performance over time using newly collected field data.
- Adding tests for longer-running service availability and recovery scenarios.

These are future testing opportunities. They are not presented as completed test results.

11 Conclusion

Testing helped verify that LeafLens works beyond the model training environment.

The backend test suite covered the main API paths, including successful predictions, invalid uploads, and monitoring instrumentation. All eight recorded backend tests passed.

The Docker Compose deployment and monitoring components were also checked during the local demonstration.

Together, these checks provide evidence that the model, API, frontend, and monitoring components work as an integrated application, while also highlighting areas that deserve further testing.